

Programovací jazyky

Co je programovací jazyk

- **programovací jazyk** = formální jazyk pro zápis algoritmů
- slouží k zadávání instrukcí počítači
- programátor pomocí něj popisuje, co má program dělat
- kód napsaný programátorem se musí převést do podoby, které rozumí procesor
- procesor rozumí pouze **strojovému kódu**
- strojový kód je tvořen instrukcemi v binární podobě

Příklady programovacích jazyků:

- C
- C++
- C#
- Java
- Python
- JavaScript
- PHP
- SQL

Struktura programovacího jazyka

Syntaxe

- **syntaxe** = pravidla zápisu programu
- určuje, jak má kód vypadat
- řeší například:
 - závorky
 - středníky
 - odsazení
 - pořadí příkazů
 - zápis podmínek a cyklů

Příklad v C#:

```
if (vek >= 18)
{
    Console.WriteLine("Plnoletý");
}
```

Příklad v Pythonu:

```
if vek >= 18:  
    print("Plnoletý")
```

Sémantika

- **sémantika** = význam programu
- říká, co zapsaný kód skutečně dělá
- program může být syntakticky správný, ale sémanticky špatný

Příklad:

```
int vek = -5;
```

- zápis může být syntakticky správný
- významově ale věk `-5` nedává smysl

Klíčová slova

- **klíčová slova** = rezervovaná slova jazyka
- mají speciální význam
- nelze je běžně použít jako název proměnné

Příklady:

- `if`
- `else`
- `while`
- `for`
- `class`
- `public`
- `return`

Úrovně programovacích jazyků

Nízkoúrovňové jazyky

- jsou blízko hardwaru
- umožňují přesnou kontrolu nad procesorem a pamětí
- jsou hůře čitelné pro člověka
- jsou závislé na konkrétní architektuře počítače

Příklady:

- strojový kód
- assembler

Vysokoúrovňové jazyky

- jsou bližší člověku
- jsou lépe čitelné
- často jsou nezávislejší na konkrétním hardwaru
- programátor nemusí řešit tolik detailů procesoru a paměti

Příklady:

- C#
- Java
- Python
- JavaScript
- PHP

Esoterické jazyky

- nejsou určené pro běžné programování
- často slouží jako experiment nebo zábava
- bývají záměrně zvláštní nebo obtížně čitelné

Příklad:

- Brainfuck

Překlad jazyka do strojového kódu

- zdrojový kód píše člověk v programovacím jazyce
- procesor ale spouští strojový kód
- proto je potřeba kód nějak zpracovat
- existují hlavní způsoby:
 - kompilace
 - interpretace
 - hybridní zpracování přes mezikód

Kompilované jazyky

- zdrojový kód se před spuštěním přeloží do strojového kódu
- překlad provádí **kompilátor**
- výsledkem může být spustitelný soubor
- program se potom spouští přímo na daném systému

- často je rychlý při běhu
- při změně kódu je potřeba program znovu zkompileovat

Výhody

- rychlý běh programu
- chyby se často odhalí už při kompilaci
- výsledný program může běžet bez zdrojového kódu

Nevýhody

- kompilace chvíli trvá
- program je často závislý na operačním systému a architektuře
- pro jinou platformu může být potřeba jiná kompilace

Příklady:

- C
- C++
- Pascal

Interpretované jazyky

- zdrojový kód se nepřekládá celý dopředu do samostatného spustitelného souboru
- program spouští **interpret**
- interpret čte kód a vykonává ho za běhu
- chyby se mohou projevit až při spuštění konkrétní části programu

Výhody

- rychlý vývoj
- snadné testování
- dobrá přenositelnost
- není vždy nutné ručně kompilovat

Nevýhody

- často pomalejší běh než u čistě kompilovaných jazyků
- program potřebuje interpret nebo runtime
- některé chyby se objeví až za běhu

Příklady:

- Python
- PHP

- JavaScript

Poznámka:

- v praxi to nebývá úplně čisté rozdělení
- například Python nebo JavaScript mohou používat různé formy mezikódu nebo JIT optimalizace

Hybridní / manažované jazyky

- kombinují kompilaci a interpretaci
- zdrojový kód se nejdříve přeloží do **mezikódu**
- mezikód není přímo strojový kód konkrétního procesoru
- mezikód spouští virtuální stroj nebo runtime
- za běhu se mezikód převádí na strojový kód

Příklady:

- C#
- Java

Mezikód

- **mezikód** = kód mezi zdrojovým kódem a strojovým kódem
- není určen přímo pro konkrétní procesor
- je určen pro virtuální stroj nebo runtime
- pomáhá s přenositelností programu mezi platformami

Příklady mezikódu:

- C# -> CIL / IL
- Java -> bytecode

JIT kompilace

- **JIT** = Just-In-Time
- převádí mezikód na strojový kód až při běhu programu
- překládá jen to, co je právě potřeba
- umožňuje optimalizace podle konkrétního prostředí

Příklad:

C# zdrojový kód → IL mezikód → CLR/JIT → strojový kód
Java zdrojový kód → bytecode → JVM/JIT → strojový kód

Multiplatformnost

- **multiplatformnost** = schopnost programu běžet na více platformách
- platforma může být:
 - operační systém
 - procesorová architektura
 - zařízení
 - runtime prostředí

Příklady platforem:

- Windows
- Linux
- macOS
- Android
- iOS

Jak se multiplatformnost řeší

1. Překlad pro každou platformu zvlášť

- typické u C nebo C++
- zdrojový kód může být stejný
- pro každou platformu se vytvoří jiný spustitelný soubor

Příklad:

```
program.exe pro Windows  
program pro Linux  
aplikace pro macOS
```

2. Virtuální stroj / runtime

- program se přeloží do mezikódu
- mezikód spustí runtime pro danou platformu
- stejný mezikód může běžet na různých systémech

Příklady:

- Java běží na JVM
- C# běží na .NET runtime / CLR

3. Interpret

- kód spouští interpret dostupný pro různé platformy
- stejný zdrojový kód může běžet na různých systémech
- interpret musí být na daném systému nainstalovaný

Příklady:

- Python
- JavaScript
- PHP

Paradigmata programování

- **programovací paradigma** = způsob přemýšlení o programu
- určuje styl, jakým se program píše
- moderní jazyky často podporují více paradigmat

Imperativní programování

- program je zapsaný jako posloupnost příkazů
- říkáme počítači krok za krokem, co má dělat
- pracuje se se změnou stavu programu

Příklad:

```
int soucet = 0;  
soucet = soucet + 5;
```

Příklady jazyků:

- C
- Pascal
- BASIC

Procedurální programování

- vychází z imperativního programování
- program se dělí na procedury nebo funkce
- používá:
 - sekvence
 - podmínky
 - cykly
 - funkce

Příklady jazyků:

- C
- Pascal

Objektově orientované programování

- program je tvořen objekty
- objekt má:
 - vlastnosti / atributy
 - metody
- objekty mezi sebou komunikují
- používá se pro modelování reálných věcí

Příklad:

```
Auto
- barva
- rychlost
- jet()
- zastavit()
```

Příklady jazyků:

- C++
- C#
- Java
- Python

Funkcionální programování

- program je založený na funkcích
- funkce zpracují vstup a vrátí výstup
- často se omezuje změna stavu
- vychází z matematiky

Příklady jazyků:

- Haskell
- F#
- Lisp

Poznámka:

- funkcionální prvky mají i běžné moderní jazyky
- například C#, JavaScript nebo Python

Deklarativní / neprocedurální přístup

- programátor popisuje, **co chce získat**
- nepopisuje přesně krok za krokem, **jak se to má provést**
- typický příklad je SQL

Příklad:

```
SELECT jmeno FROM Studenti WHERE trida = '4.A';
```

- říkáme, jaká data chceme
- neřešíme přesný postup vyhledání

Syntaxe podle vzhledu jazyka

C-derived syntaxe

- syntaxe odvozená od jazyka C
- používá:
 - složené závorky
 - středníky
 - kulaté závorky u podmínek a funkcí

Příklady:

- C
- C++
- C#
- Java
- JavaScript
- PHP

Příklad:

```
if (x > 0)
{
    Console.WriteLine(x);
}
```

Odsazovací syntaxe

- struktura programu je dána odsazením
- nepoužívají se složené závorky pro bloky
- nutí k čitelnému formátování

Příklad:

- Python

```
if x > 0:
```

```
print(x)
```

Keyword-based syntaxe

- používá klíčová slova pro začátek a konec bloků
- například `begin/end` nebo `do/end`

Příklady:

- Pascal
- Delphi
- Lua

Architektura .NET

- **.NET** je vývojová platforma od Microsoftu
- slouží k vývoji různých typů aplikací
- nejčastěji se používá s jazykem **C#**
- umožňuje vývoj:
 - desktopových aplikací
 - webových aplikací
 - mobilních aplikací
 - her
 - cloudových služeb
 - API

.NET Framework

- starší platforma .NET
- primárně určená pro Windows
- používá se u starších aplikací
- dnes se pro nové projekty častěji používá moderní .NET

.NET Core / moderní .NET

- novější platforma
- multiplatformní
- běží na:
 - Windows
 - Linux
 - macOS
- je rychlejší a modernější než původní .NET Framework
- od verze .NET 5 se používá označení jednoduše **.NET**

Mono

- open-source implementace .NET
- umožňovala běh .NET aplikací i mimo Windows
- významná byla hlavně pro:
 - Linux
 - mobilní platformy
 - Xamarin

Základní části .NET architektury

Csharp (C#)

- hlavní jazyk používaný v .NET
- vysokoúrovňový jazyk
- staticky typovaný
- objektově orientovaný
- podporuje i další paradigmatata

Kompilátor

- překládá zdrojový kód C# do mezikódu
- výsledkem je **IL**
- IL znamená **Intermediate Language**
- někdy se používá také označení **CIL**

Zjednodušený průběh překladu v .NET:

C# kód → kompilátor → IL/CIL → CLR → JIT → strojový kód

CLR

- **CLR** = Common Language Runtime
- běhové prostředí .NET
- spouští programy napsané pro .NET
- stará se například o:
 - běh programu
 - správu paměti
 - garbage collector
 - bezpečnost
 - výjimky
 - JIT kompilaci

JIT kompilátor

- převádí IL na strojový kód
- děje se při běhu programu
- výsledný strojový kód už spouští procesor

IL → JIT → strojový kód

BCL / knihovny

- **BCL** = Base Class Library
- základní knihovna tříd v .NET
- obsahuje hotové třídy a funkce
- například pro:
 - práci se soubory
 - kolekce
 - textové řetězce
 - datum a čas
 - síťovou komunikaci

Využití .NET

ASP.NET

- framework pro webové aplikace a API
- používá se pro backend webových aplikací

MAUI / Xamarin

- frameworky pro mobilní a multiplatformní aplikace
- umožňují psát aplikace v C#

Unity

- herní engine
- pro skriptování používá C#